

Learning machines

SPRFST ships the pieces you need to train a small model without reaching for anything else: an array type with the usual operations in `std.tensor`, and layers, a network and a trainer in `std.ai`.

Tensors

A tensor is a shape and a flat list of numbers.

```
use std.io
use std.tensor

fn main() {
  let a = tensor.make([2, 3], [1.0, 2.0, 3.0, 4.0, 5.0, 6.0])
  let b = tensor.make([3, 2], [7.0, 8.0, 9.0, 10.0, 11.0, 12.0])

  io.say("shape      {tensor.shape(a)}")
  io.say("a + a      {tensor.to_list(tensor.add(a, a))}")
  io.say("a * 2      {tensor.to_list(tensor.scale(a, 2.0))}")
  io.say("a @ b      {tensor.to_list(tensor.matmul(a, b))}")
  io.say("a^T       {tensor.to_list(tensor.transpose(a))}")
  io.say("mean       {tensor.mean(a)}")
  io.say("softmax    {tensor.to_list(tensor.softmax(tensor.make([3], [1.0, 2.0, 3.0]))
)})")
  io.say("argmax     {tensor.argmax(tensor.make([4], [0.1, 0.9, 0.3, 0.2]))}")
}
```

The full set: `make zeros random shape at put add sub mul scale matmul transpose relu sigmoid tanh softmax sum mean argmax to_list dot`.

A network

`std.ai` builds on those. A `Dense` layer holds its own weights; a `Network` chains two of them and trains by gradient descent.

```
use std.io
use std.ai
use std.rand
use std.math

fn main() {
  rand.seed(11)

  var net = ai.Network { rate: 0.6 }
  net.build(2, 6, 1)           ~~ 2 inputs, 6 hidden, 1 output

  let inputs = [[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]]
  let wanted = [[0.0], [1.0], [1.0], [0.0]]

  let final_loss = net.train(inputs, wanted, 2000, false)
  io.say("loss after training: {final_loss}")
}
```

```

var correct = 0
for i in 0..4 {
    let got = net.predict(inputs[i])
    if math.round(got[0]) == wanted[i][0] { correct = correct + 1 }
}
io.say("{correct} of 4 correct")
}

```

XOR is the classic test because no single layer can learn it. If the loss stops falling, the usual causes are a learning `rate` that is too large, too few hidden units, or a seed that started somewhere awkward — `rand.seed` makes every run repeatable so you can tell the difference.

The parts

Piece	What it does
<code>ai.Dense</code>	one fully connected layer: start, forward
<code>ai.Network</code>	two layers plus build, predict, learn, train
<code>ai.apply</code> / <code>ai.slope</code>	activations and their derivatives
<code>ai.loss</code>	mean squared error
<code>ai.tokenize</code>	text to word tokens
<code>ai.vocabulary</code>	tokens to numbers

Activations available to a layer: `relu`, `sigmoid`, `tanh`, and `linear` for anything else.

Text in, numbers out

```

use std.io
use std.ai

fn main() {
    let words = ai.tokenize("SPRFST is fast. SPRFST is small. Fast is good!")
    io.say("tokens      {words}")

    let vocab = ai.vocabulary(words)
    io.say("vocabulary {vocab}")

    var encoded: [Int] = []
    for w in words { encoded.push(vocab.get(w) ?? 0) }
    io.say("encoded    {encoded}")
}

```

That is the front of every language model: split, number, feed.

Where the edges are

Be clear eyed about what this is. The gradients in `std.ai` are written out by hand for the two layer case; there is no automatic differentiation, no GPU, and no pretrained weights. It is enough to learn a function from a few thousand examples, to run inference from weights you load yourself, and to understand

exactly what every number is doing—and `std.ai` is ordinary SPRFST source in `std/ai.spf`, so a third layer or a new optimiser is a file you can edit.

Anything larger belongs in a program that calls out to a dedicated runtime, and the extension points in chapter 27 are how you would attach one.

Exercise

Train the network to tell even numbers from odd ones: feed it the four bits of a number from 0 to 15 and expect 1 for even. Then print the cases it gets wrong.